

Experiment – 1

Negative of an Image

AIM: To simulate and display an image, extract its color components, convert it into grayscale, and generate its negative (binary and grayscale).

Theory

Image negative transformation, also known as image inversion or complement, is a fundamental point operation in digital image processing that reverses the brightness values of an image. For greyscale images with 8-bit depth (pixel values ranging from 0 to 255), the negative transformation is performed using the formula:

$$s = L - 1 - r$$

where, s = output pixel value

L = maximum gray level (255 for 8-bit images)

r = input pixel value

This transformation converts dark pixels (low values) to bright pixels (high values) and vice versa, creating a photographic negative effect. For colour images, each RGB channel is inverted independently, causing colour inversions (e.g., red becomes cyan, green becomes magenta). The negative transformation is particularly useful in medical imaging and remote sensing applications, where it helps reveal details hidden in darker regions of the original image by enhancing contrast

Code

```
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

img = Image.open("./content/fox.jpg")
imgarr = np.array(img)

R = imgarr[:, :, 0]
G = imgarr[:, :, 1]
B = imgarr[:, :, 2]

gray = np.mean(imgarr, axis=2).astype(np.uint8)
neg_gray = 255 - gray
threshold = 128
binary = (gray > threshold) * 255
binary_neg = 255 - binary

plt.figure(figsize=(12, 8))
plt.subplot(2, 4, 1)
plt.imshow(img)
plt.title('Original')
plt.axis('off')
```

```
plt.subplot(2, 4, 2)
plt.imshow(R, cmap='Reds')
plt.title('Red')
plt.axis('off')

plt.subplot(2, 4, 3)
plt.imshow(G, cmap='Greens')
plt.title('Green')
plt.axis('off')

plt.subplot(2, 4, 4)
plt.imshow(B, cmap='Blues')
plt.title('Blue')
plt.axis('off')

plt.subplot(2, 4, 5)
plt.imshow(gray, cmap='gray')
plt.title('Grayscale')
plt.axis('off')

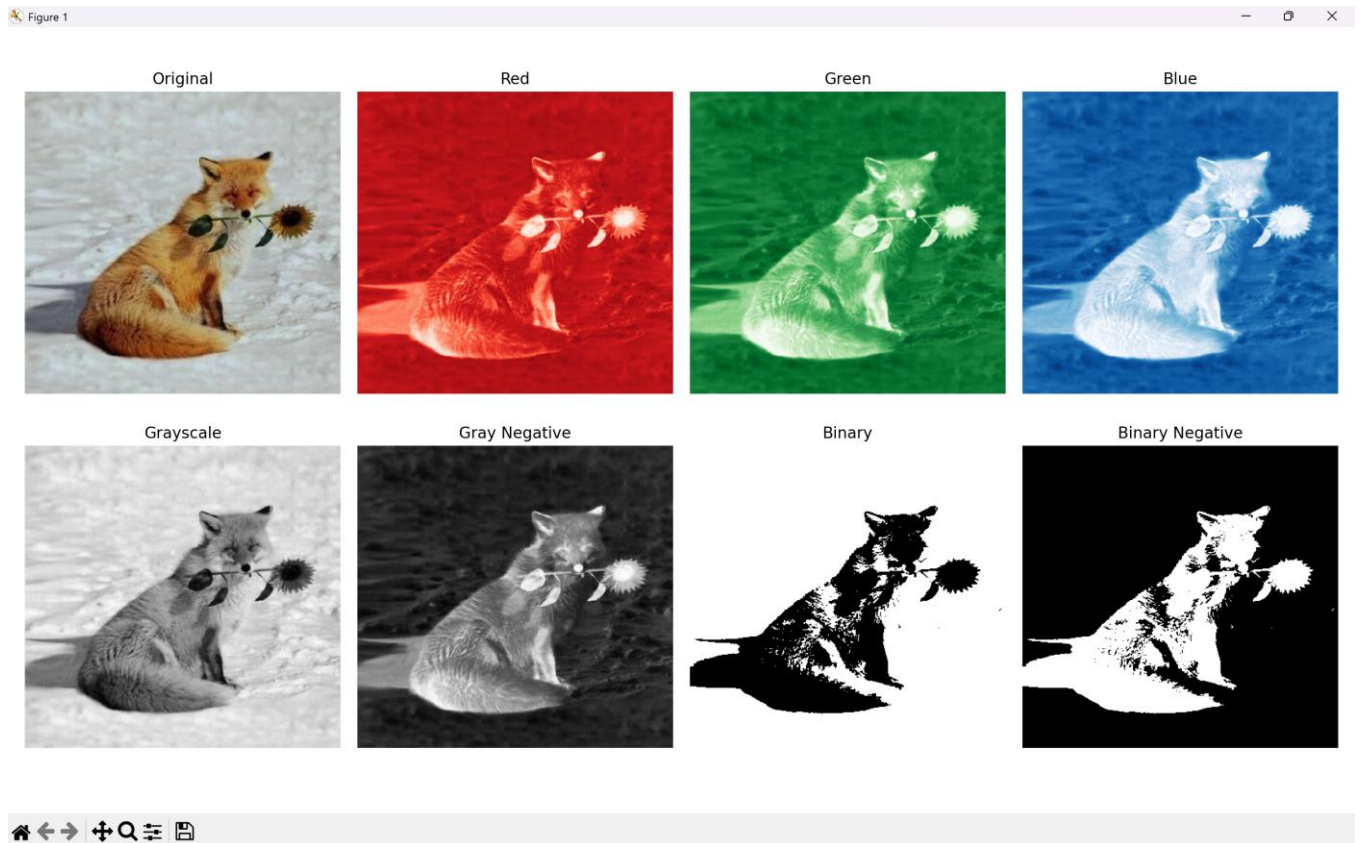
plt.subplot(2, 4, 6)
plt.imshow(neg_gray, cmap='gray')
plt.title('Gray Negative')
plt.axis('off')

plt.subplot(2, 4, 7)
plt.imshow(binary, cmap='gray')
plt.title('Binary')
plt.axis('off')

plt.subplot(2, 4, 8)
plt.imshow(binary_neg, cmap='gray')
plt.title('Binary Negative')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Output



Results and Discussion

The program successfully computed the negative of the input image by subtracting each pixel value from 255. The transformation effectively reversed the intensity values, making dark regions appear bright and bright regions appear dark. The negative image clearly demonstrates the inversion effect, areas that were dark in the original image (low pixel values) now appear bright (high pixel values) in the negative, and vice versa. This transformation is useful for enhancing details in underexposed regions, medical imaging applications, and artistic effects. The implementation using NumPy array operations is computationally efficient and handles both grayscale and color images correctly, as each channel is inverted independently.